Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251211**
Nama Lengkap	**Jendri Mansamnaf Japen**
Minggu ke / Materi	**06 / Struktur Kontrol Perulangan**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

MATERI 6.3

6.3.1 Definisi Perulangan

Jalannya suatu program dapat diatur secara sekuensial, percabangan, perulangan, maupun kombinasi dari ketiganya. Pengaturan tersebut biasa disebut sebagai struktur kontrol. Perulangan digunakan apabila dalam program diperlukan untuk:

• Melakukan suatu hal yang sama beberapa kali.

• Melakukan suatu hal secara bertahap, di mana setiap tahap sebenarnya memiliki langkah yang sama.

• Mengakses sekumpulan data dalam suatu struktur data, misalnya: List, Tuple, Queue, Stack dan beberapa struktur data lainnya.

Pada Python, perulangan dapat dilakukan dengan menggunakan for, while maupun dengan cara rekursif. Untuk pertemuan ini, dibahas mengenai perulangan for dan while

6.3.2 Bentuk Perulangan for

Dalam Python, perulangan dapat dilakukan menggunakan for dan while. Perulangan for biasanya digunakan ketika jumlah perulangan sudah diketahui sejak awal atau ketika melakukan operasi yang sama pada rentang nilai tertentu.

Perulangan for sering menggunakan fungsi range() untuk menghasilkan urutan angka. Bentuk penggunaan range() antara lain:

1. range(stop) => menghasilkan angka dari 0 sampai stop-1.
2. range(start, stop, step) => menghasilkan angka dari start sampai stop-1 dengan langkah tertentu.

Contoh penggunaan for adalah menampilkan angka dari 1 sampai 100 dengan range(1,101). Variabel dalam perulangan biasanya digunakan sebagai counter untuk menghitung jumlah perulangan.

Jika nilai counter tidak diperlukan, dapat menggunakan _ sebagai pengganti variabel, misalnya untuk menampilkan teks Hello World sebanyak 100 kali.

Selain itu, fungsi range() juga dapat menggunakan step negatif untuk melakukan perulangan secara mundur. Misalnya:

1. range(2,101,2) untuk menampilkan bilangan genap dari 2 sampai 100.
2. range(100,1,-2) untuk menampilkan bilangan genap dari 100 sampai 2 secara menurun.

6.3.3 Bentuk Perulangan While

Perulangan while digunakan ketika jumlah perulangan belum diketahui sebelumnya. Perulangan ini akan terus berjalan selama kondisi yang diberikan masih bernilai True, dan akan berhenti ketika kondisi tersebut menjadi False.

Secara umum bentuk perulangan while adalah:

```
Start

while kondisi:

    operasi

    operasi

    step
```

Contoh penggunaan while adalah untuk memastikan pengguna memasukkan bilangan genap. Program akan terus meminta input hingga pengguna memasukkan angka yang memenuhi syarat (bilangan genap). Jika pengguna memasukkan bilangan ganjil, program akan meminta input kembali sampai bilangan genap dimasukkan.

```
>>> bilangan = 0
... genap = False
...
... while genap == False:
...     bilangan = int(input("Masukkan bilangan genap: "))
...
...     if bilangan % 2 == 0:
...         genap = True
...
... print(bilangan, "yang anda masukkan adalah bilangan genap")
...
Masukkan bilangan genap:
```

Gambar 6.0.1 contoh kode program

Perulangan while sangat cocok digunakan ketika jumlah perulangan tidak diketahui, misalnya saat program harus terus meminta input dari pengguna sampai kondisi tertentu terpenuhi.

6.3.5 Konversi dari Bentuk for Menjadi Bentuk while

Perulangan for pada Python sebagian besar dapat dikonversi menjadi perulangan while. Kedua jenis perulangan ini memiliki fungsi yang sama, yaitu untuk menjalankan suatu perintah secara berulang hingga mencapai kondisi tertentu.

```
>>> for i in range(1, 6):
...     print(i)
...
1
2
3
4
5
>>> |
```

Gambar 6.1.1 Contoh perulangan menggunakan for

Dalam sebuah perulangan terdapat beberapa komponen penting yang harus ada agar perulangan dapat berjalan dengan baik. Komponen tersebut adalah nilai awal, nilai akhir, dan langkah (step). Nilai awal digunakan untuk memulai perulangan, nilai akhir digunakan sebagai batas berhenti perulangan, dan step digunakan untuk mengubah nilai counter pada setiap iterasi.

```
>>> for i in range(1, 11, 2):
...     print(i)
...
1
3
5
7
9
>>>
```

Gambar 6.2.1 Contoh penggunaan nilai awal, akhir, dan step

Pada perulangan for, nilai awal, nilai akhir, dan langkah biasanya sudah diatur melalui fungsi range(). Fungsi ini akan menghasilkan deret angka sesuai dengan parameter yang diberikan.

```
>>> for i in range(1, 11):
...     print(i)
```

Gambar 6.3.1 contoh penggunaan fungsi range()

Program tersebut akan menampilkan bilangan dari 1 sampai 10 karena range(1,11) berarti mulai dari 1 dan berhenti sebelum 11.

Sementara itu, pada perulangan **while**, nilai awal, kondisi akhir, dan langkah harus dituliskan secara lebih jelas oleh programmer. Nilai awal biasanya dideklarasikan sebelum perulangan dimulai, sedangkan perubahan nilai dilakukan di dalam perulangan.

```
>>> i = 1
... while i <= 10:
...     print(i)
...     i = i + 1
```

Gambar 6.4.1 Contoh perulangan menggunakan while

```
>>> i = 1
... while i <= 10:
...     print(i)
...     i = i + 1
...
1
2
3
4
5
6
7
8
9
10
>>>
```

Gambar 6.5.1 contoh perulangan menggunakan while jika di jalankan

Sebagai contoh, perulangan for yang menampilkan angka dari 1 sampai 10 dapat dikonversi menjadi perulangan while dengan menentukan nilai awal i = 1, kondisi perulangan i ≤ 10, dan langkah perubahan i = i + 1 pada setiap iterasi. Dengan demikian, kedua bentuk perulangan tersebut sebenarnya memiliki fungsi yang sama, hanya berbeda pada cara penulisannya.

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

SOAL 6.1

```
[4]: def perkalian(a, b):
         hasil = 0
         proses= ""

         for i in range(a):
             hasil += b
             proses += str(b)
             if i < a - 1:
                 proses += " + "

         print(f"{a} x {b} = {proses} = {hasil}")

     if __name__ == "__main__":
         a = int(input("Masukan angka pertama: "))
         b = int(input("Masukan angka kedua: "))

         perkalian(a, b)
```

```
Masukan angka pertama:  6
Masukan angka kedua:  5
6 x 5 = 5 + 5 + 5 + 5 + 5 + 5 = 30
```

Program pada gambar di atas menunjukkan bahwa operasi perkalian dapat dilakukan dengan metode penjumlahan berulang. Program menjumlahkan bilangan kedua sebanyak nilai bilangan pertama menggunakan perulangan hingga diperoleh hasil akhir. Selain itu, program juga menampilkan proses penjumlahan sehingga pengguna dapat memahami bagaimana hasil perkalian tersebut diperoleh.

SOAL 6.2

```
[6]: #latihan 6.2
def ganjil(bawah, atas):
    hasil = []

    if bawah < atas:
        for i in range(bawah, atas + 1):
            if i % 2 != 0:
                hasil.append(str(i))
    else:
        for i in range(bawah, atas - 1, -1):
            if i % 2 != 0:
                hasil.append(str(i))
    print(",".join(hasil))

if __name__ == "__main__":
    bawah = int(input("Masukan batas bawah: "))
    atas = int(input("Masukan batas atas: "))

    ganjil(bawah, atas)
```

```
Masukan batas bawah:  10
Masukan batas atas:  30
11,13,15,17,19,21,23,25,27,29
```

Program pada gambar di atas digunakan untuk menampilkan deret bilangan ganjil berdasarkan batas bawah dan batas atas yang dimasukkan oleh pengguna. Jika batas bawah lebih kecil dari batas atas, maka bilangan ganjil akan ditampilkan dari kecil ke besar. Sebaliknya, jika batas bawah lebih besar dari batas atas, maka bilangan ganjil akan ditampilkan dari besar ke kecil menggunakan perulangan. Dengan demikian, program ini menunjukkan penggunaan fungsi, percabangan, dan perulangan untuk menghasilkan deret bilangan ganjil sesuai dengan rentang yang diberikan.

SOAL 6.3

```
[11]: #latihan 6.3
def hitung_ips(jumlah_mk):
    total_bobot = 0

    for i in range(1, jumlah_mk + 1):
        nilai = input(f"Nilai MK {i}: ").upper()

        if nilai =="A":
            bobot = 4
        elif nilai =="B":
            bobot = 3
        elif nilai == "C":
            bobot = 2
        elif nilai == "D":
            bobot = 1
        else:
            bobot = 0

        total_bobot += bobot

    ips = total_bobot / jumlah_mk
    print("Nilai IPS anda semester ini %.2f" % ips)

if __name__ == "__main__":
    jumlah = int(input("Berapa jumlah mata kuliah? "))
    hitung_ips(jumlah)
```

```
Berapa jumlah mata kuliah?  6
Nilai MK 1:  A
Nilai MK 2:  B
Nilai MK 3:  C
Nilai MK 4:  A
Nilai MK 5:  D
Nilai MK 6:  C
Nilai IPS anda semester ini 2.67
```

Program pada gambar di atas digunakan untuk menghitung Indeks Prestasi Semester (IPS) berdasarkan jumlah mata kuliah dan nilai yang diperoleh mahasiswa. Setiap nilai huruf (A, B, C, D) akan dikonversi menjadi bobot angka, kemudian dijumlahkan untuk mendapatkan total bobot. Setelah itu, total bobot dibagi dengan jumlah mata kuliah sehingga diperoleh nilai IPS. Program ini menunjukkan penggunaan fungsi, perulangan, percabangan, dan input pengguna untuk melakukan perhitungan nilai IPS secara otomatis.

Link github: https://github.com/japenjendri-design/Praktikkum-Algoritma-dan-Pemrograman.git